Arrays, Linked Lists & Paradigmen

Scala – sauber strukturierte und lesefreundlich formatierte Fassung der bereitgestellten Notizen

Inhalt

1. Imports • 2. Exceptions • 3. Try - Catch • 4. Exceptions werfen • 5. Arrays • 6. Initialisierung • 7. Zugriff & Manipulation • 8. Weitere Methoden • 9. Lineare Verarbeitung • 10. Bisektion • 11. Divide and conquer • 12. Selectionsort (Minimalsortiren) O(n^2) • 13. Insertionsort (Sortieren durch Einfügen) O(n^2) • 14. Quicksort O(n*log2(n)) • 15. Mergesort O(n*log2(n)) • 16. tests • 17. Greedy-Prinzip • 18. Slice-Funktion für Arrays (Kombination zweier Arrays) • 19. a1 • 20. Backtracking • 21. Zwei Möglichkeiten: (1) Gegenstand aufnehmen, (2) Gegenstand ignorieren • 22. **Ausgabe der Lösungen** • 23. Dynamische Programmierung • 24. Schritt 1: DP-Tabelle füllen • 25. Schritt 2: Rückverfolgung der gewählten Gegenstände • 26. Pointer • 27. value --> Wert, next --> Speichert Referenz auf nachfolgendes Objekt • 28. Gibt Kopie zurück • 29. Queue - generisch • 30. Test

Imports

```
import scala.reflect.ClassTag
```

Exceptions

```
/*
```

Try - Catch

```
try
  {...}
catch
  case e1:NumberFormatException => {...}
  case e2:ArithmeticException => {...}
  case e3:IndexOutOfBoundsException => {...}
  case e4 => {...} //alle anderen Fälle --> case _
finally
  {...}
```

Exceptions werfen

```
throw new Exception("...")
throw new ArithmeticException("...")
*/
```

Arrays

```
/*
i) Datenstrukur fester Länge
ii) Speichern Variablen
iii) Alle Elemente gelten als gleichwertig & sind vom gleichen Datentyp
iv) Indizierter Zugriff Möglich Arr(0) --> 1. Element im Array, Arr(4) --> 5. Elem
var name:Array[Datentyp] = new Array[Datentyp](Anzahl Felder)
 */
```

Initialisierung

```
var arr1:Array[Int] = Array.ofDim[Int](5) //Leeres Array auf Basis der Größe und Datentyp der Eingabedaten
for (i <- arr1.indices){ //Durchlauf des Arrays
  arr1(i) = i //Manuelle Wertezuweisung
}
```

```
arr1 //Array(0, 1, 2, 3, 4)

var arr2 = Array(3,0,1,4,2) //Werte Direkt zuweisen

var arr3 = Array.fill(5)(7) //(n)(wert) --> Array der Länge n mit gleichen Werten --> Array(7, 7, 7, 7, 7)

var arr4 = Array.range(1,4) //(start, end) --> Array(1,2,3)

var arr5 = Array.tabulate(5)(i => if (i%2==0) i*2 else 0) //(n)(f) --> Array(0, 0, 4, 0, 8, 0)

var arr62d:Array[Array[Int]] = Array.ofDim(3,3) //3x3 Matrix --> Array[Array[Int]] = Array(Array(0, 0, 0), Array(0
var n = 0
for (i <- arr62d.indices){
  for (j <- arr62d.indices){ //oder for (i <- arr62d.indices; j <- arr62d.indices){...}
    arr62d(i)(j) = n //i = Zeile, j = Spalte
    n += 1
  }
}
arr62d(1) //Array(3,4,5)
```

Zugriff & Manipulation

```
val elemFirstarr1 = arr1(0) //0
val elemLastarr1 = arr1(4) //4

arr1(0) = 1 //arr1 = Array(1,1,2,3,4)

print(arr1.mkString("(", ", ", ")")) //(1, 1, 2, 3, 4)

for (elem <- arr1){
  print(elem) //11234
}

for (row <-arr62d){
  println(row.mkString("(", ", ", ")")) //Ausgabe arr62d als 3x3 Matrix
}
```

Weitere Methoden

```
val sumarr1 = arr1.sum  //Summe der Elemente --> Array(1,1,2,3,4) => 11
val maxarr1 = arr1.max //maximum --> 4
val minarr1 = arr1.min //minimum --> 1

val sortarr2:Array[Int] = arr2.sorted  //Gibt eine neue sortierte Kopie des Arrays zurück --> Array(3, 0, 1, 4, 2)

val revarr1:Array[Int] = arr1.reverse  //Array(4, 3, 2, 1, 1)

val evenarr1:Array[Int] = arr1.filter(_ % 2 == 0) //Filter: Gerade Zahlen --> Array(4, 2)
val larger4arr1:Array[Int] = arr1.filter(_ > 4) //Filter: Gerade Zahlen --> Array()

val doublearr1:Array[Int] = arr1.map(_ * 2) //Mapping: Jede Zahl verdopppeln --> Array(2, 2, 4, 6, 8)

val arr1con5:Boolean = arr1.contains(5) //Elem entahlten ? --> false

val listarr1:List[Int] = arr1.toList //List(1, 1, 2, 3, 4)
```

Lineare Verarbeitung

```
/*
Lineare Verarbeitung --> Elementweise Verarbeitung
Für n Elemente Laufzeit von n
 */
def linearSearchArray(arr:Array[Int], x:Int): Boolean =
  var i = 0
  var found = false
  while (i < arr.length-1 && found != true){
    if (arr(i) == x){
      found = true
    }
    i += 1
  }
```

```
  return found
```

Bisektion

```
/*
Problemgröße wird halbiert --> Laufzeit log2(n)
Grenzen bilden und Suchebereich anpassen
Daten müssen sortiert sein
*/
def binarySearchArray(arr:Array[Int], x:Int): Boolean =
  var left = 0
  var right = arr.length-1
  var found = false
  while (left <= right){
    val mid:Int = (left + right)/2
    if (x < arr(mid)){
      right = mid-1
    } else if (x > arr(mid)){
      left = mid+1
    } else {
      found = true
    }
  }
  return found
```

Divide and conquer

```
/*
Lineare Verarbeitung: Aufteilung der Datenmenge in zwei (oder mehr) Untermenge (möglichst gleich groß)
Bearbeitung:
  1. Divide:  Aufteilung vornehmen
  2. Conquer: Lösung der Teile via Rekursion
  3. Merge:   Kombination der Teillösungen zur Gesamtlösung
Beispiele:
  1. Quicksort
  2. Mergesort
*/
```

Selectionsort (Minimalsortiren) O(n^2)

```
def selectionsortArray(arr:Array[Int]): Array[Int] =
  var arrSorted:Array[Int] = arr.clone()
  for (i <- 0 to arrSorted.length-2){
    var cacheIndex = i
    for (j <- i+1 to arrSorted.length-1){
      if (arrSorted(j) < arrSorted(cacheIndex)){
        cacheIndex = j
      }
    }
    val tmp = arrSorted(i)
    arrSorted(i) = arrSorted(cacheIndex)
    arrSorted(cacheIndex) = tmp
  }
  return arrSorted
```

Insertionsort (Sortieren durch Einfügen) O(n^2)

```
def insertionsortArray(arr:Array[Int]): Array[Int] =
  var arrSorted = arr.clone()
  for (i <- 0 to arrSorted.length-2){
    var current = arrSorted(i+1)
    for (j <- 0 to i){
      if (current < arrSorted(j)){
        val tmp = arrSorted(j)
        arrSorted(j) = current
        current = tmp
      }
    }
    arrSorted(i+1) = current
  }
```

```
  return arrSorted
```

Quicksort O(n*log2(n))

```
def quicksortArray(arr:Array[Int]): Array[Int] =
  if (arr.length <= 1){
    return arr
  }
  var arrSorted = arr.clone()
  val pivot = arrSorted(0)
  val left = arrSorted.filter(_ < pivot)
  val mid = arrSorted.filter(_ == pivot)
  var right = arrSorted.filter(_ > pivot)
  arrSorted = quicksortArray(left) ++ mid ++ quicksortArray(right)
  return arrSorted
```

Mergesort O(n*log2(n))

```
def merge(left:Array[Int], right:Array[Int]):Array[Int] =
  var i = 0
  var j = 0
  var n = 0
  var merged:Array[Int] = Array.ofDim(left.length + right.length)
  while (i < left.length && j < right.length) {
    if (left(i) < right(j)) {
      merged(n) = left(i)
      n += 1
      i += 1
    } else {
      merged(n) = right(j)
      n += 1
      j += 1
    }
  }
  while(i < left.length){
    merged(n) = left(i)
    n += 1
    i += 1
  }
  while (j < right.length) {
    merged(n) = right(j)
    n += 1
    j += 1
  }
  return merged
def mergesortArray(arr:Array[Int]): Array[Int] =
  if (arr.length <= 1){
    return arr
  }
  var arrSorted = arr.clone()
  val midIndex = arrSorted.length/2
  val left = arrSorted.take(midIndex) //midIndex Einträge
  val right = arrSorted.drop(midIndex) //minIndex Einträge werden ausgelassen
  arrSorted = merge(mergesortArray(left), mergesortArray(right))
  return arrSorted
```

tests

```
val us1 = Array(1, 2, 3, 4, 5)
val us2 = us1.reverse
val us3 = Array(3, 2, 1, 5, 4)
val us4 = Array(2, 4, 1, 3, 5)
println("Slectionsort:")
selectionsortArray(us1)
selectionsortArray(us2)
selectionsortArray(us3)
selectionsortArray(us4)
println("Insertionsort")
insertionsortArray(us1)
insertionsortArray(us2)
insertionsortArray(us3)
```

```
insertionsortArray(us4)
println("Quicksort")
quicksortArray(us1)
quicksortArray(us2)
quicksortArray(us3)
quicksortArray(us4)
println("Mergesort")
mergesortArray(us1)
mergesortArray(us2)
mergesortArray(us3)
mergesortArray(us4)
```

Greedy-Prinzip

```
/*
Diese Algorithmen arbeiten nach einem Prinzip, bei dem jede nächste zu treffende Entscheidung bestmöglich (z.B. un
Dadruch wird zu einem bestimmten Problem (sofern möglich) eine Lösung gefunden, aber nicht zwingend die beste
--> Entscheidungen können nicht rückgängig gemacht werden

Beispiel, BackPack:
Einbrecher wählt Wertgegenstände(Eigenschaften: Gewicht, Wert), kann aber nur eine gewisse Menge aufgrund einer Tr
Greedy-Prinzip:
1. Gegenstände nach Wert sortieren
2. Immer den wertvollsten passenden Gegenstand auswählen
3. Wiederhole bis Trage-Kapazität erreicht
*/
type Gegenstand = (String, Int, Int) //Name, Wert, Gewicht
def ekWert(g1:Gegenstand, g2:Gegenstand): Boolean = (g1._1 < g2._1)
def egWert(g1:Gegenstand, g2:Gegenstand): Boolean = (g1._1 > g2._1)
def glWert(g1:Gegenstand, g2:Gegenstand): Boolean = (g1._1 == g2._1)
```

Slice-Funktion für Arrays (Kombination zweier Arrays)

```
def slice[T: ClassTag](a1:Array[T], a2:Array[T]): Array[T] =
  var ret:Array[T] = Array.ofDim[T](a1.length + a2.length)
  var n = 0
```

a1

```
for (i <- a1){
    ret(n) = i
    n += 1
  }
  for (j <- a2){
    ret(n) = j
    n+= 1
  }
  return ret

def quicksortArray[T: ClassTag](arr:Array[T])(ek:(T, T) => Boolean)(eg:(T, T) => Boolean)(gl:(T, T) => Boolean): A
  if (arr.length <= 1) return arr
  val pivot: T = arr(0)
  val left: Array[T] = arr.filter(ek(_, pivot))
  val mid: Array[T] = arr.filter(gl(_, pivot))
  val right: Array[T] = arr.filter(eg(_, pivot))
  slice(slice(quicksortArray(left)(ek)(eg)(gl), mid), quicksortArray(right)(ek)(eg)(gl))

def kanpSackGreedy(g:Array[Gegenstand], capacity:Int): List[Gegenstand] =
  var backPack:List[Gegenstand] = Nil
  var carriedW:Int = 0
  val gSorted = quicksortArray[Gegenstand](g)(ekWert)(egWert)(glWert)
  for (i <- gSorted){
    if (carriedW + i._3 < capacity) {
      backPack = backPack ::: i :: Nil
      carriedW += i._3
    }
  }
  return backPack

val test1 = Array(
  ("Goldbarren", 100, 10),
```

```
  ("Silberbarren", 60, 20),
  ("Diamant", 150, 5),
  ("Laptop", 80, 7),
  ("Kamera", 70, 6),
  ("Uhr", 50, 4),
  ("Münzen", 40, 3)
)
val kapazitaet1 = 30

val test2 = Array(
  ("Kleines Buch", 5, 1),
  ("Notizblock", 8, 2),
  ("Kugelschreiber", 3, 1),
  ("Handy-Ladegerät", 10, 2),
  ("Sonnenbrille", 12, 2),
  ("Portemonnaie", 15, 3),
  ("Schlüssel", 7, 1)
)
val kapazitaet2 = 7

val l1:List[Gegenstand] = kanpSackGreedy(test1, kapazitaet1)
val l2:List[Gegenstand] = kanpSackGreedy(test2, kapazitaet2)
```

Backtracking

```
/*
Bruteforce; Durchprobieren aller Möglichkeiten
Falls entscheidung nicht zur Lösung führt, wird sie rückgängig gemacht
Findet alle Lösungen
Hohe Laufzeit --> Exponentiell
*/
def knapSackBackTrAlg(g: Array[Gegenstand], index: Int, capacityCurrent: Int, capacity: Int, path: List[Gegenstand
  if (capacityCurrent > capacity) return Nil // Falls Gewicht überschritten wurde -> ungültig
  if (index >= g.length) return List(path) // Alle Gegenstände getestet -> Rückgabe der aktuellen Lösung
```

Zwei Möglichkeiten: (1) Gegenstand aufnehmen, (2) Gegenstand ignorieren

```
val withItem = knapSackBackTrAlg(g, index + 1, capacityCurrent + g(index)._3, capacity, path :+ g(index))
  val withoutItem = knapSackBackTrAlg(g, index + 1, capacityCurrent, capacity, path)

  withItem ++ withoutItem // Alle möglichen Lösungen sammeln
}

def knapSackBackTrStart(g: Array[Gegenstand], capacity: Int): List[List[Gegenstand]] = {
  knapSackBackTrAlg(g, 0, 0, capacity, List())
}

val loesungen = knapSackBackTrStart(test1, kapazitaet1)
```

Ausgabe der Lösungen

```
println("Alle möglichen Rucksack-Lösungen:")
for (i <- loesungen.indices){
  println(s"Lösung $i: ${loesungen(i)}")
}
```

Dynamische Programmierung

```
/*
Vermeidet doppelte Berechnungen, indem Teilergebnisse gespeichert werden
Kann Probleme lösen, die Rekursiv exponentiell wären
Optimierungsprobleme mit optimalen Teilproblemen

Beispiel, BackPack:
Tabelle mit Teillösungen wird angelegt, um Berechnungen zu ersparen
*/
def knapSackDP(g:Array[Gegenstand], capacity: Int): (Int, List[Gegenstand]) = {
  val n = g.length
```

```
  val dp = Array.ofDim[Int](n + 1, capacity + 1)
```

Schritt 1: DP-Tabelle füllen

```
for (i <- 1 to n; w <- 0 to capacity) {
    val (name, value, weight) = g(i - 1)
    if (weight <= w) {
      dp(i)(w) = Math.max(dp(i - 1)(w), dp(i - 1)(w - weight) + value)
    } else {
      dp(i)(w) = dp(i - 1)(w)
    }
  }
```

Schritt 2: Rückverfolgung der gewählten Gegenstände

```
var w = capacity
  var resValue = dp(n)(capacity)
  var chosenItems = List[Gegenstand]()

  for (i <- n until 0 by -1 if resValue > 0) {
    val (name, value, weight) = g(i - 1)
    if (resValue != dp(i - 1)(w)) {  // Item wurde aufgenommen
      chosenItems = (name, value, weight) :: chosenItems
      resValue -= value
      w -= weight
    }
  }
  (dp(n)(capacity), chosenItems)
}
```

Pointer

```
/*
Werden Dateneinträge in den Speicher geschrieben, kann via Referenz auf diese Daten verwiesen werden. Die Referenz
Die Referenz dient als Pointer, über den die Daten adressiert, manipuliert, ... werden können.
Auf gleiche Art und Weise können verkettete Listen/Bäume/Etc. erstellt werden, welche in sich auf ein nächstes Obj
Für die Traversierung ist es wichtig, einen zusätzlichen Pointer zu erstellen, da die bestehende Referenz für ein
Daten im Speicher, die nicht adressiert werden können, werde während der Laufzeit von einem "Garbage Collecter" en
*/
```

value --> Wert, next --> Speichert Referenz auf nachfolgendes Objekt

```
class Elem(var value: Int = 0, var next: Elem = null){

  def mkString(start:String = "(", delimiter:String = ", ", end:String = ")"): String =
    var s = start
    var current = this
    while (current.next != null) {
      s = s + current.value.toString + delimiter
      current = current.next
    }
    s = s + current.value.toString + end
    return s

  def copy(): Elem =
    if (this == null) return null
    var c = new Elem(this.value, null)
    var p = this.next
    var q = c
    while (p != null) {
      print(p.value.toString + " ")
      q.next = new Elem(p.value, null)
      p = p.next
      q = q.next
    }
    return c

  def intersec(l:Elem): Elem =
```

```
    var t = this
    var ltmp = l
    var ret = new Elem()
    while (t != null && ltmp != null) {
      if (t.value > ltmp.value) {
        ltmp = ltmp.next
      }
      else if (t.value < ltmp.value) {
        t = t.next
      }
      else {
        ret.next = new Elem(t.value, null)
      }
    }
    return ret.next

  def head(): Int =
    return this.value

  def tail(): Elem =
```

Gibt Kopie zurück

```
var dummy = new Elem()
    var pointer = this
    var pointerRes = dummy
    while(pointer != null){
      pointerRes.next = new Elem(pointer.value)
      pointerRes = pointerRes.next
      pointer = pointer.next
    }
    return dummy.next.next

  def length(): Int =
    var i = 0
    var p = this
    while (p != null){
      i += 1
      p = p.next
    }
    return i

  def append(w: Int): Unit = {
    var p = this
    while (p.next != null){
      p = p.next
    }
    p.next = new Elem(w, null)
  }
}

var tList:Elem = new Elem(2, null)
tList = new Elem(4, tList)
tList = new Elem(9, tList)
tList = new Elem(13, tList)
tList = new Elem(15, tList)

val List1:Elem = null
val List2 = Elem()

val s =  tList.mkString()
val neu = tList.copy()
neu.mkString()
neu.length()
neu.tail().mkString()
```

Queue - generisch

```
class Queue[T](var v: T, var n: Queue[T])

def isEmpty[T](q: Queue[T]): Boolean = {
  return q == null
}
```

```
def enque[T](q: Queue[T], x: T): Queue[T] = {
  if (q == null){
    return new Queue[T](x, null)
  }
  var p = q
  while (p.n != null){
    p = p.n
  }
  p.n = new Queue[T](x, null)
  return q
}

def deque[T](q: Queue[T]): Queue[T] = {
  if (q == null){
    throw new Exception("Q ist leer")
  }
  return q.n
}

def first[T](q: Queue[T]): T = {
  if (q == null){
    throw new Exception("Q ist leer")
  }
  return q.v
}

def contains[T](q: Queue[T], x: T): Boolean = {
  var p = q
  while (p != null){
    if (p.n == x){
      return true
    }
    p = p.n
  }
  return false
}

def queueToString[T](q: Queue[T]): String = {
  var p = q
  var s: String = ""
  while (p != null){
    s = s + s"${p.v} <- "
    p = p.n
  }
  return s
}
```

Test

```
var qInt = new Queue[Int](0,null)
qInt = enque(qInt, 3)
qInt = enque(qInt, 7)
qInt = enque(qInt, 5)
contains(qInt, 9)
contains(qInt, 7)
first(qInt)
queueToString(qInt)
qInt = deque(qInt)
queueToString(qInt)
first(qInt)
qInt = deque(qInt)
queueToString(qInt)
qInt = deque(qInt)
queueToString(qInt)
qInt = deque(qInt)
isEmpty(qInt)
deque(qInt)
```